
HungerHeal

Running it locally, step by step

13 steps, cold machine to a matched donation

Verified on Windows with Docker Desktop

Event-driven microservices · Go · Kafka · Redis

HungerHeal — running it locally

Step by step, from a cold machine to a donation being matched on screen. Every command here was run and verified on this Windows machine.

The short version, once everything is set up:

1. start Docker Desktop
2. `docker compose up -d`
3. open `http://localhost:8080`

The rest of this document is the detail behind those three lines.

What you need installed

Tool	Needed?	Why
Docker Desktop	Yes	Every service, database and broker runs in a container
Git for Windows	Yes	Provides Git Bash, which runs the topic script
Node 20+	Only for frontend hot-reload	Not needed to run the project
Go	Only to run the Go unit tests	The Go services compile <i>inside</i> Docker

You do not need to install MongoDB, Redis, Kafka or Cassandra. Docker runs all of them.

Which terminal to use

Most commands are identical in **PowerShell** and **Git Bash**. Where they differ, both are given. Two Windows-specific traps:

Trap 1 — `bash` in PowerShell is WSL, not Git Bash. Typing `bash scripts/create-topics.sh` in PowerShell launches `C:\Windows\System32\bash.exe`, which is **WSL** — a different Linux environment that may not see Docker at all. Either run the script from a Git Bash terminal, or call Git Bash by its full path (shown in step 5).

Trap 2 — `curl` in PowerShell is not real `curl`. In Windows PowerShell 5.1, `curl` is an alias for `Invoke-WebRequest` and prints a wall of headers. Type `curl.exe` instead and you get the real thing in either shell.

The easiest option: in VS Code, open a terminal and pick **Git Bash** from the dropdown next to the `+`. Then every command below works as written.

First time only

1 Start Docker Desktop

Open **Docker Desktop** from the Start menu and wait until the bottom-left corner says **Engine running**.

This is the step that will catch you out most often. Docker Desktop does not start by itself after a reboot, and every command below fails with something like:

```
failed to connect to the docker API at npipe:////./pipe/dockerDesktopLinuxEngine
```

That error always means *Docker Desktop is not running* — nothing is wrong with the project.

To stop it happening: **Docker Desktop → Settings → General → tick "Start Docker Desktop when you sign in to your computer."**

2 Open a terminal in the project folder

```
cd "C:\Users\Hrishikesh\OneDrive\Desktop\HungerHeal-ADV-"
```

3 Check the `.env` file exists

Your `.env` already exists and is complete — **you do not need to do anything here**. It holds your real JWT secret and OpenCage API key.

On a **fresh clone** on another machine, create it from the template:

```
cp .env.example .env
```

(`cp` works in both PowerShell and Git Bash.) The template works unchanged — nothing has to be filled in. The only mandatory value is `JWT_SECRET`, and the template already has a valid placeholder.

4 Build and start everything

```
docker compose up -d --build
```

Flag	Means
<code>up</code>	create and start every container
<code>-d</code>	<i>detached</i> — run in the background and give you the terminal back
<code>--build</code>	build the images from source first

The first build takes several minutes — two Go services compile, six Node services install their dependencies, and the frontend bundle is built. Every run after that takes about 30 seconds.

This starts **13 containers**: seven core services, the gateway, the frontend, MongoDB, Redis, Kafka and Kafka UI.

5 Create the Kafka topics

Kafka would auto-create a topic the first time something publishes to it — but with the default of **one** partition. The system is designed for **three**, so create them properly once.

In Git Bash:

```
bash scripts/create-topics.sh
```

In PowerShell (calls Git Bash by its full path, avoiding the WSL trap):

```
& "C:\Program Files\Git\bin\bash.exe" scripts/create-topics.sh
```

You should see eight topics listed:

```
donation.created      donation.assigned    donation.accepted
donation.rejected     donation.timeout     donation.collected
donation.unassigned   agent.location.updated
```

The script is safe to re-run — it skips topics that already exist. Once done, **you never need to run it again** unless you wipe the data with `down -v`.

6 Check everything is healthy

```
curl.exe http://localhost:4000/ready
```

Wait until it says `"status":"ready"` with all seven services `"up"`:

```
{ "status": "ready", "services": {
  "auth-service": "up", "donation-service": "up",
  "geocoding-service": "up", "agent-location-service": "up",
  "assignment-engine": "up", "tracking-service": "up",
  "notification-service": "up" } }
```

If any say `"down"`, wait 20 seconds and try again — services report ready only once they have connected to MongoDB, Redis and Kafka. Measured on this machine: **about 30 seconds** from `up` to all-ready.

7 Open the app

<http://localhost:8080>

Use `localhost` — not your machine's IP address. The browser only allows location access on `localhost` or HTTPS, and agents cannot go on shift without it.

Every time after that

1. Start Docker Desktop wait for "Engine running"
2. `docker compose up -d` ~30 seconds, no `--build` needed
3. `curl.exe http://localhost:4000/ready`
4. Open `http://localhost:8080`

Only add `--build` when you have **changed source code** — see "After changing code" below.

Trying it out — see a donation get matched

8 Open two separate browser windows

- a **normal** window — this will be the **donor**
- an **incognito** / **private** window — this will be the **collector**

You need two because the login lives in the browser's local storage, and one browser profile can only hold one login at a time.

9 Set up the collector FIRST

This order matters. A donation is only matched to someone **already on shift** nearby. Post first and it correctly ends up *"No agent available yet"* — the system working as designed, not a bug.

In the **incognito** window:

1. **Get started** → choose **"I collect food"**
2. Tick **"Insulated bag or box"**
3. Under categories, make sure **cooked prepared** is selected
4. Create the account — you land on the dashboard
5. Click **Go on shift** → **Allow** when the browser asks for location
6. **Wait** until the panel changes from *"Finding your location..."* to **"On shift"** and shows your coordinates

Step 6 matters: until your first GPS fix arrives, the server has not heard from you and you cannot be matched.

10 Post a donation as the donor

In the **normal** window:

1. **Get started** → choose **"I donate food"** → create the account
2. Fill in the donation form: - **Title:** *Leftover biryani from a wedding* - **Category:** *Cooked / prepared meals* - **Quantity:** 40 servings
3. In the map, click **Use my location** → allow
4. **Post donation**

Why "Use my location": it puts the pickup at the same spot as the collector, guaranteeing they are in range. (Your OpenCage key means typed addresses anywhere also work — without a key, only ten Bengaluru landmarks would resolve.)

11 Accept it as the collector

Switch to the **incognito** window. Within about **five seconds** a **collection request** appears with a live countdown and *"Ranked #1 for this pickup"*.

Click **Accept**. It moves to **To collect**.

Back in the donor window, the donation now reads **"On the way"** with the collector's phone number.

12 Mark it collected

In the collector window, click **Mark collected**. The donor window updates to **Collected** and the impact figures increase.

13 See why that collector was chosen (optional)

Open a **third** window (or log out of one) → **Get started** → "**I want to observe**". The dashboard shows every donation — click one to see the **full score breakdown** for every candidate: distance, category fit, load and rating, each weighted.

There is no assign button there. That is deliberate.

Optional extras

Analytics and Cassandra

Off by default, because Cassandra is heavy. To include them:

```
docker compose --profile analytics up -d
```

That adds two containers (15 total). Cassandra takes a minute or two to become healthy on first start.

Kafka UI — watch the events

http://localhost:8090 → **Topics** → `donation.created` or `donation.assigned` → **Messages**. Every event the system publishes is visible here.

Frontend hot-reload (only if editing the UI)

```
cd frontend  
npm install          # first time only  
npm run dev
```

Opens on **http://localhost:5173** and reloads as you edit. It talks to the same backend containers, so `docker compose up -d` must still be running.

Stopping

Command	Effect
<code>docker compose down</code>	stops everything — your data is kept
<code>docker compose down -v</code>	stops everything and deletes all data : users, donations, history
<code>docker compose --profile analytics down</code>	as above, including Cassandra if you started it

After `down -v`, re-run **step 5** (the topics) next time you start.

Closing Docker Desktop also stops everything, but `docker compose down` is the clean way.

After changing code

Rebuild only the service you changed:

```
docker compose up -d --build donation-service
```

Service names are the ones in `docker-compose.yml`: `auth-service`, `donation-service`, `geocoding-service`, `agent-location-service`, `assignment-engine`, `tracking-service`, `notification-service`, `api-gateway`, `frontend`.

Useful commands

Want to	Command
See what is running	<code>docker compose ps</code>
Follow one service's logs	<code>docker compose logs -f assignment-engine</code>
Last 50 lines of a service	<code>docker compose logs --tail 50 tracking-service</code>
Restart one service	<code>docker compose restart tracking-service</code>
Follow one donation everywhere (Git Bash)	<code>docker compose logs \\\ grep <traceId></code>
Same, in PowerShell	<code>docker compose logs \\\ Select-String <traceId></code>
Run the end-to-end test	<code>bash scripts/smoke-gateway.sh</code> (Git Bash)

The **traceId** for any donation is shown on its monitoring detail view. One id follows it through every service's logs.

Troubleshooting

Symptom	Cause and fix
<code>failed to connect to the docker API / npipe error</code>	Docker Desktop is not running. Start it, wait for <i>Engine running</i>
<code>/ready</code> shows a service "down"	Still starting. Wait 20s. If it persists: <code>docker compose logs <service></code>
<code>/ready</code> returns nothing at all	The gateway is not up. <code>docker compose ps</code> — is <code>hh-gateway</code> listed?
"No agent available yet"	No collector on shift nearby. Do step 9 before step 10, and use Use my location in both
Collector never gets an offer	Panel still says " <i>Finding your location...</i> " — wait for the coordinates. Or cooked prepared was not ticked at signup
Location permission never asked	You opened an IP address instead of <code>localhost</code> . Use <code>http://localhost:8080</code>
<code>bash</code> opens a Linux prompt	That is WSL. Use Git Bash, or the full-path command in step 5
<code>curl</code> prints pages of headers	PowerShell's alias. Use <code>curl.exe</code>
<code>port is already allocated</code>	Something else is using 8080, 4000 or 27018. Close it, or stop other Docker projects
Changed code but nothing different	You need <code>--build</code> for that service — see "After changing code"
A page looks stale after rebuild	Hard refresh: Ctrl + Shift + R

Ports in use

Port	What
8080	the app (frontend)
4000	api-gateway
4001–4008	the individual services
8090	Kafka UI
27018	MongoDB (27017 is taken by a local mongod on this machine)
6379	Redis
29092	Kafka, from the host
9042	Cassandra (only with the analytics profile)